

The Frontier Gazette

VOL. 1, NO. 048

2026-03-30

FRONT PAGE

If I were CEO of Bluesky

Scripting News

I've written a bunch of pieces with this premise, what if I were CEO of Apple being the first, in January 1996, before Steve Jobs returned. This time I'm writing as if I were CEO of Bluesky, a company that just got a new interim CEO, Toni Schneider, formerly of Automattic, the company behind WordPress. This started as a comment in reply to Colin Devroe on Mastodon, but quickly exceeded its 500 character limit. And no doubt I will expand on it over the course of the day.

Here's the issue. AT Proto is proposing to be a better web than the collection of standards that make up the web in 2026, starting with HTTP and HTML and DNS and including Markdown, WebSockets, MP3 and RSS and probably a few others. Maybe they can come up with something better organized and with more consistent interfaces. But the web doesn't work that way. Once it embraces a method of doing something, it goes on, and doesn't reconsider. It's exactly like evolution in the natural world.

Example: RSS was a deeply entrenched competitor when Atom came along, intending to do everything RSS did but do it differently and better. It did get some support and still does to this day, but the differences are flattened out, most feed-reading software doesn't know if the news came from RSS or Atom, the distinction is buried in low-level code.

If you were to look at the size of the developer base for the web, it would be clear how steep a hill AT Proto has to climb, and why? What's in it for Bluesky except satisfaction of ego? Not a good business proposition for a startup.

But they can't abandon the developers who made a bet on AT Proto, so they should give it to a standards body, work with them, but at the same time work on interop with products like WordPress and support inbound and outbound RSS. Markdown would be nice too. Get rid of the character limit and support links, styling, enclosures (for podcasting) and make their posts editable. In other words they have some catching up to do re the web. That's where their leadership would be welcome instead of questioned.

Colin, I don't think they should do it for you and me, they should do it as an investment in their future. Get in the game. The idea of creating something that stands alone is imho very un-web, and not differentiated from their competition. The web was made for small companies like Bluesky. Trying to act like a giant in a way even the biggest giants wouldn't work is not a formula for success. I think Toni and Matt would understand this.

It's exactly like evolution in the natural world.

Scripting News

FEATURES

How GitHub engineers tackle platform problems

Fabian Aguilar Gomez · GitHub Engineering

In my spare time I enjoy building Gundam models, which are model kits to build iconic mechas from the Gundam universe. You might be wondering what this has to do with software engineering. Product engineers can be seen as the engineers who take these kits and build the Gundam itself. They are able to utilize all pieces and build a working product that is fun to collect or even play with!

Platform engineers, on the other hand, supply the tools needed to build these kits (like clippers and files) and maybe even build a cool display so everyone can see the final product. They ensure that whoever is constructing it has all the necessary tools, even if they don't physically build the Gundam themselves.

About a year ago, my team at GitHub moved to the infrastructure organization, inheriting new roles and Areas of Responsibility (AoRs). Previously, the team had tackled external customer problems, such as building the new deployment views across environments. This involved interacting with users who depend on GitHub to address challenges within their respective industries. Our new customers as a platform engineering team are internal, which makes our responsibilities different from the product-focused engineering work we were doing before.

Going back to my Gundam example, rather than constructing kits, we're now responsible for building the components of the kits. Adapting to this change meant I had to rethink my approach to code testing and problem solving.

Whether you're working on product engineering or on the platform side, here are a few best practices to tackle platform problems.

Understanding your domain

One of the most critical steps before tackling problems is understanding the domain. A "domain" is the business and technical subject area in which a team and platform organization operate. This requires gaining an understanding of technical terms and how these systems interact to provide fast and reliable solutions. Here's how to get up to speed:

Talk to your neighbors: Arrange a handover meeting with a team that has more knowledge and experience with the subject matter. This meeting provides an opportunity to ask questions about terminology and gain a deeper understanding of the problems the team will be addressing.

Investigate old issues: If there is a backlog of issues that are either stale or still persistent, they may give you a better understanding of the system's current limitations and potential areas for improvement.

Read the docs: Documentation is a goldmine of knowledge that can help you understand how the system works.

Bridging concepts to platform-specific skills

While the preceding advice offers general guidance applicable to both product and platform teams, platform teams — serving as the foundational layer — necessitate a more in-depth understanding.

Networks : Understanding network fundamentals is crucial for all engineers, even those not directly involved in network operations. This includes concepts like TCP, UDP, and L4 load balancing, as well as debugging tools such as `dig`. A solid grasp of these areas is essential to comprehend how network traffic impacts your platform.

Operating systems and hardware: Selecting appropriate virtual machines (VMs) or physical hardware is vital for both scalability and cost management. Making well-informed choices for particular applications requires a strong grasp of both. This is closely linked to choosing the right operating system for your machines, which is important to avoid systems with vulnerabilities or those nearing end of life.

Infrastructure as Code (IaC): Automation tools like Terraform, Ansible, and Consul are becoming increasingly essential. Proficiency in these tools is becoming a necessity as they significantly decrease human error during infrastructure provisioning and modifications.

Distributed systems : Dealing with platform issues, particularly in distributed systems, necessitates a deep understanding that failures are inevitable. Consequently, employing proactive solutions like failover and recovery mechanisms is crucial for preserving system reliability and preventing adverse user experiences. The optimal approach for this depends entirely on the specific problem and the desired system behavior.

Knowledge sharing

By sharing lessons and ideas, engineers can introduce new perspectives that lead to breakthroughs and innovations. Taking the time to understand why a project or solution did or didn't work and sharing those findings provides new perspectives that we can use going forward.

Here are three reasons why knowledge sharing is so important:

Teamwork makes the dream work: Collaboration often results in quicker problem resolution and fosters new solution innovation, as engineers have the opportunity to learn from each other and expand upon existing ideas.

Prevent lost knowledge : If we don't share our lessons learned, we prevent the information from being disseminated across the team or organization. This becomes a problem if an engineer leaves the company or is simply unavailable.

Improve our customer success: As engineers, our solutions should effectively serve our customers. By sharing our

Election 2012: Electoral combinations

NPR Apps Blog

This is the first in a series of two (or more) blog posts about how we built the Swing State Scorecard and our Election 2012 results site .

The idea

Early in the development of the Swing State Scorecard we determined that we wanted to tell a story about how many combinations (2-state, 3-state) of tossup states there are which would win the election for Obama or Romney (based on NPR projections). One idea that seemed compelling was to try to actually illustrate all the possible combinations of states that would win the election for each candidate. Doing so would, we speculated, demonstrate very clearly how important certain states (Florida) were to each candidate's overall chance of winning the election. We had seen one other example of this, but it was difficult to use and didn't allow you to quickly compare the candidates.

Building it

The one wrinkle to generating our “paths to victory” was that we weren't actually sure we had the computational capacity to do it. Pre-rendering all the possible combinations would be a huge pain—the numbers quickly became unmanageable and an API would have been untenable on election night. The better option was to actually generate the combinations on the fly in Javascript, but we weren't sure whether or not the average user's computer would have the horsepower to do it.

With these constraints in mind I set to work prototyping the algorithms that generate the combinations. Javascript lacks good library support for this sort of operation, but I was able to find many combination generating functions on Stack Overflow and elsewhere. The vast majority of these were recursive solutions, which immediately blew the call stack in several of our target browsers. Fortunately, I found a very fast, non-recursive solution developed by Stack Overflow user Sid_M . I modified this very slightly and the final function is called in combinations in our codebase. To my surprise this method of generating combinations is very, very fast and works great even in IE. Of course, performance of the algorithm does degrade quickly with the number of tossup states, so we had to keep that number under thirteen in order for this method to work.

Pruning the combinations

The resulting combinations still needed to be pruned down in order to be interesting. We filtered them in two ways. First (1), and most obviously, we only included ones that accumulated enough votes to form a winning combination. (This would probably have been faster if we pushed the logic down into the combinations algorithm, but I preferred to keep things well-factored.) Secondly (2), we removed any combination which was a superset of a previous combination. That is, if we already had the combination “Florida

+ Colorado”, then we discarded “Florida + Colorado + New Hampshire”. Fortunately the output of our combinations algorithm was sorted, so we were able to do all this pruning in a single iteration over the list.

Here is the final code that generates and prunes the combinations for the Scorecard:

Obama's lead in the polls shrank (or even reversed, depending on who you read) after we developed this approach, but we felt the illustration of the relative complexity of the paths to victory remained compelling. For election night we refactored this code into a “prediction mode” that would kick on automatically when we got down to the last twelve states.

As it turns out the election was over so quickly many users probably never even noticed it, but had the ballot counting gone on into Wednesday it would have provided a ongoing way for users to interact with the results. Apparently, we weren't the only ones with this in mind as the New York Times published a different view on the same information with their Paths to the White House app just before the election and updated automatically it throughout the night.

Beginner Guide to CJ Affiliate (Commission Junction) in 2022

Unknown · TechCrunch

In this CJ Affiliate guide, I will share with you everything you need to get started on the platform, I will give you an in-depth look at the network and how it works.

You will learn how to earn money with the platform. If you're not interested, I'll share some of the best CJ affiliate programs and alternatives. By the end of this post, I will also answer some of the FAQs on the platform and give my quick CJ review.

Sounds Good So let's start

What is CJ Affiliate?

Commission Junction is an online advertising company that offers affiliate programs for various retailers. Since 1998, it has been known as one of the oldest and most popular affiliate networks.

Commission Junction has consistently ranked among the top 10 affiliate networks

With in-depth data analysis and an unmatched understanding of clients needs, CJ has established itself as a leader in performance marketing.

For Advertiser:

CJ provides advertisers with a variety of tracking, management, and payment options. As an affiliate network, CJ can help you launch multiple affiliate programs from a centralized network.

CJ's experienced team of account managers is available to help at every step—from program set-up to optimization.

For Publishers:

CJ offers a variety of well-paying affiliate programs. You can find affiliate programs in almost every niche at CJ. With CJ, you can also find promotional tools such as banners and product feeds, which help you promote your website.

The reporting tools are unparalleled and provide granular data that can assist you in fine-tuning your campaigns for maximum results.

First things first, CJ is free to join! If you are new to the world of affiliate marketing, don't worry—you'll be able to join right away.

The requirements for joining CJ are almost similar to other networks. For example, you must have a blog or social media follower.

Isn't it obvious? Let's explore the details.

High-quality, unique content.

Non-gated content, of course.

Site privacy policy.

No software, coupon/deal, or incentive models

Your traffic must be from US and Canada.

The main traffic source must not be paid

10K+ monthly traffic on your website.

How Does CJ Affiliate Work?

Between advertisers and publishers, CJ Affiliate acts as a middleman.. Advertisers sign up on CJ to promote their products or services, while publishers sign up on CJ to find and join affiliate programs to make money. CJ then tracks the sales or leads generated by the publisher and pays them a commission according to the terms of the affiliate program.

CJ provides a win-win situation for both sides: advertisers get more sales and publishers make money.

In order to free up both parties to concentrate on their job, CJ also handles payments and other technical issues. Now that you know how CJ works, let's learn more about how to sign up and start making money with it.

How to Start Making Money Online Using CJ Affiliate

To get started using CJ's affiliate network, you'll need to register for an account. To do this, you must have a website or social media profile with relevant content and an audience from the US or Canada.

Create a CJ account, complete the application process, and then wait for approval. You shouldn't worry CJ is not strict as other network in approving applications.

Here's how you can signup for CJ

Visit cj.com to register as a publisher.

Fill in information such as your nationality, email, password, and more.

Verify your email address now.

You will be transferred right away to your CJ Account Manager, which contains crucial data including network statistics and performance summaries.

Now from the Account menu, head to Network Profile. And to sign up for any affiliate programme offered on CJ, complete this profile separately.

When applying for CJ, you need to share these two pieces of information

Creating Combine-compatible versions of async/await-based APIs

Swift by Sundell

A challenge that many developers face as they maintain various code bases over time is how to neatly connect different frameworks and APIs in a way that properly adheres to the conventions of each technology involved.

For example, as teams around the world are starting to adopt Swift 5.5's `async/await` -powered concurrency system, we'll likely find ourselves in situations where we need to create versions of our `async` -marked APIs that are compatible with other asynchronous programming techniques — such as Combine.

While we've already taken a look at how Combine relates to concurrency APIs like `async` sequences and streams, and how we can make it possible to call `async` -marked functions within a Combine pipeline — in this article, let's explore how we could make it easy to create Combine-based variants of any `async` API, regardless of whether it was defined by us, by Apple, or as part of a third-party dependency.

Async futures

Let's say that an app that we're working on contains the following `ModelLoader`, which can be used to load any Decodable model over the network. It performs its work through an `async` function that looks like this:

```
class ModelLoader { ...
```

Now let's say that we'd also like to create a Combine-based version of the above `loadModel` API, for example in order to be able to call it within specific parts of our code base that might've been written in a more reactive style using the Combine framework.

We could of course choose to write that sort of compatibility code specifically for our `ModelLoader` type, but since this is a general problem that we're likely to encounter multiple times when working with Combine-based code, let's instead create a more generic solution that we'll be able to easily reuse across our code base.

Since we're dealing with `async` functions that either return a single value, or throw an error, let's use Combine's `Future` publisher to wrap those calls. That publisher type was specifically built for these kinds of use cases, since it gives us a closure that can be used to report a single `Result` back to the framework.

So let's go ahead and extend the `Future` type with a convenience initializer that makes it possible to initialize an instance with an `async` closure:

```
extension Future where Failure == Error { convenience init
(operation: @escaping () async throws -> Output) { self
.init { promise in Task { do { let output = try await operation
() promise (. success (output)) } catch { promise (. failure
(error)) } } } }
```

For more information on how Combine's `Future` type works, check out "Using Combine's futures and subjects".

The power of creating an abstraction like that, which isn't tied to any specific use case, is that we'll now be able to apply it to any `async` API that we want to make Combine-compatible. All it takes is a few lines of code that calls the API that we're looking to bridge within a closure that's passed to our new `Future` initializer — like this:

Neat! Note how we could've chosen to give that Combine-based version the same `loadModel` name that our `async` -powered one has (since Swift supports method overloading). However, in this case, it might be a good idea to clearly separate the two, which is why the above new API has a name that explicitly includes the word "Publisher".

Reactive async sequences

`Async` sequences and streams are perhaps the closest that the Swift standard library has ever come to adopting reactive programming, which in turn makes those APIs behave very similarly to Combine — in that they enable us to emit values over time.

In fact, in the article "Async sequences, streams, and Combine", we took a look at how Combine publishers can even be directly converted into `async` sequences using their `values` property — but what if we wanted to go the other way, and convert an `async` sequence (or stream) into a publisher?

Continuing with the `ModelLoader` example from before, let's say that our loader class also offers the following API, which lets us create an `AsyncThrowingStream` that emits a series of models loaded from an array of URLs:

```
class ModelLoader { ...
```

For an example of a concrete implementation of an API just like the one above, check out the aforementioned article "Async sequences, streams, and Combine".

Just like before, rather than rushing into writing code that specifically converts the above `loadModels` API into a Combine publisher, let's instead try to come up with a generic abstraction that we'll be able to reuse whenever we want to write similar bridging code elsewhere within our project.

This time, we'll extend Combine's `PassthroughSubject` type, which gives us complete control over when its values are emitted, as well as when and how it should terminate. However, we're not going to model this API as a convenience initializer, since we want to make it clear that calling this API will, in fact, make the created subject start emitting values right away. So let's make it a static factory method instead — like this:

```
Task { do { for try await value in sequence { subject. send
(value) }
```

```
subject. send (completion: . finished ) } catch { subject. send
(completion: failure (error)) } }
```

Everything our app template does: July 2014 edition

NPR Apps Blog

The NPR News Apps team, before its merger with the Multimedia team to form Visuals, made an early commitment to building client-side news applications, or static sites. The team made this choice for many reasons — performance, reliability and cost among them — but such a decision meant we needed our own template to start from so that we could easily build production-ready static sites. Over the past two years, the team has iterated on our app template, our “opinionated project template for client-side apps.” We also commit ourselves to keeping that template completely open source and free to use.

We last checked in on the app template over a year ago. Since then, our team has grown and merged with the Multimedia team to become Visuals. We have built user-submitted databases, visual stories and curated collections of great things, all with the app template. As we continue to build newer and weirder things, we learn more about what our app template needs. When we develop something for a particular project that can be used later — say, the share panel from Behind the Civil Rights Act — we refactor it back into the app template. Since we haven’t checked in for a while, I thought I would provide a rundown of everything the app template does in July 2014.

The fundamental backbone of the app template is the same as it has always been: a Flask app that renders the project locally and provides routes for baking the project into flat files. All of our tooling for local development revolves around this Flask app. That includes:

Fabric : Using the app template requires knowledge of the command line. This is because we use Fabric, a Python library for running functions from the command line, to automate every task from bootstrapping the template to deploying to production.

Jinja2 : Jinja2 provides templating within HTML, which is essential for baking out our various pages within an app. The Flask app allows us to pass any data we want, but we pass in data from a Google Spreadsheet and data from a configuration file by default (more on this later).

awscli : All of our apps are hosted on Amazon S3. It is cheap, reliable and fast. With awscli and Fabric, we can fully automate deployment from the command line. It is true one-touch deployment.

Out of these tools, we essentially built a basic static site generator. With just these features, the app template wouldn’t be all that special or worth using. But the app template comes with plenty more features that make it worth our investment.

Copytext and Google Spreadsheets

A few months ago, we released copytext, a library for accessing a spreadsheet as a native Python object suitable

for templating. Some version of copytext has been a part of the app template for much longer, but we felt it was valuable enough to factor out into its own library.

We often describe our Google Docs to Jinja template workflow entirely as “copytext”, but that’s not entirely true. Copytext, the library, works with a locally downloaded .xlsx version of a Google spreadsheet (or any .xlsx file). We have separate code in the app template itself that handles the automated download of the Google Spreadsheet.

Once we have the Google Spreadsheet locally, we use copytext to turn it into a Python object, which is passed through the Flask app to the Jinja templates (and a separate JS file if we want to render templates on the client).

The benefits of using Google Spreadsheets to handle your copy are well-documented. A globally accessible spreadsheet lets us remove all content from the raw HTML, including tags for social media and SEO. Spreadsheets democratize our workflow, letting reporters, product owners and copy editors read through the raw content without having to dig into HTML. Admittedly, a spreadsheet is not an optimal place to read and edit long blocks of text, but this is the best solution we have right now.

Another piece of the backbone of the static site generator is the render pipeline. This makes all of our applications performance-ready once they get to the S3 server. Before we deploy, the render pipeline works as follows:

Compile our LESS files into CSS files.

Compile our JavaScript templates (JSTs).

When running through the Jinja templates, some more optimization magic happens.

NPR Apps Blog

Render our app configuration file and copy spreadsheet as JavaScript objects.

Run through the Flask routes and render Jinja templates into flat HTML as appropriate.

When running through the Jinja templates, some more optimization magic happens. We defined template tags that allow us to “push” individual CSS and JavaScript files into one minified and compressed file. You can see the code that creates those tags here. In production, this reduces the number of HTTP requests the browser has to make and makes the files the browser has to download as small as possible.

Sensible Front-End Defaults

We like to say that the app template creates the 90% of every website that is exactly the same so we can spend our time perfecting the last 10%, the presentation layer.

A few bytes here, a few there, pretty soon you're talking real memory

Dave Cheney

Today's post comes from a recent Go pop quiz. Consider this benchmark fragment. 1

```
func BenchmarkSortStrings(b *testing. B) { s :=
[]string{"heart", "lungs", "brain", "kidneys", "pancreas"} b.
ReportAllocs() for i := 0; i
```

 Escaping to the heap

To make the example a little more explicit, here's the benchmark rewritten without the sort. Strings helper function:

```
func BenchmarkSortStrings(b *testing. B) { s :=
[]string{"heart", "lungs", "brain", "kidneys", "pancreas"} b.
ReportAllocs() for i := 0; i
```

 Size classes

To explain what a size class is, consider how a theoretical Go runtime could allocate 24 bytes on its heap. A simple way to do this would be keep track of all the memory allocated so far using a pointer to the last allocated byte on the heap. To allocate 24 bytes, the heap pointer is incremented by 24, and the previous value returned to the caller. As long as the code that asked for 24 bytes never writes beyond that mark this mechanism has no overhead. Sadly, in real life, memory allocators don't just allocate memory, sometimes they have to free it.

Eventually the Go runtime will have to free those 24 bytes, but from the runtime's point of view, the only thing it knows is the starting address it gave to the caller. It does not know how many bytes after that address were allocated. To permit deallocation, our hypothetical Go runtime allocator would have to record, for each allocation on the heap, its length. Where are the allocation for those lengths allocated? On the heap of course.

In our scenario, when the runtime wants to allocate memory, it could request a little more than it was asked for and use it to store the amount requested. For our slice example, when we asked for 24 bytes, we'd actually consume 24 bytes plus some overhead to store the number 24. How large is this overhead? It turns out the minimum amount that is practical is one word. 5

To record a 24 byte allocation the overhead would 8 bytes. 25% is not great, but not terrible, and as the size of the allocation goes up, the overhead would become insignificant. However, what would happen if we wanted to store just one byte on the heap? The overhead would be eight times the amount of data we asked for! Is there are more efficient way to allocate small amounts on the heap?

Rather than storing the length alongside each allocation, what if all the thing of the same size were stored together? If all the 24 byte things are stored together, then the runtime would automatically know how large they are. All the runtime needs is a single bit to indicate if a 24 byte area is in use or not. In Go these areas are known as size classes, because all the things of the same size are stored together (think school class—all the students are the same grade—not a C++

class). When the runtime needs to allocate a small amount, it does so using the smallest size class that can accomodate the allocation.

Unlimited size classes for all

Now we know how size classes work, the obvious question is, where are they stored? Not surprisingly, the memory for a size class comes from the heap. To minimise overhead, the runtime allocates a larger amount from the heap (usually a multiple of the system page size) then dedicates that space for allocations of a single size. But, there's a problem.

The pattern of allocating a large area to store things of the same size works well 6 if there's a fixed, preferably small, number of allocation sizes, but in a general purpose language programs can ask the runtime for an allocation of any size. 7

For example, imagine asking the runtime for 9 bytes. 9 bytes is an uncommon size, so it's likely that a new size class for things of size 9 would be required. As 9 byte things are uncommon, it's likely the remainder of the allocation, 4kb or more, would be wasted. Because of this the set of possible size classes is fixed. If a size class of the exact amount is not available, the allocation is rounded up to the next size class. In our example 9 bytes might be allocated in the 12 byte size class. The overhead of 3 unused bytes is better than an entire size class allocation which goes mostly unused.

All together now

This is the final piece of the puzzle. Go 1.15 did not have a 24 byte size class, so the heap allocation of ss was allocated in the 32 byte size class. Thanks to the work of Martin Möhrmann Go 1.16 has a 24 byte size class, which is a perfect fit for slice values assigned to interfaces.

In Go these areas are known as size classes, because all the things of the same size are stored together (think school class—all the students are the same grade—not a C++ class).

Dave Cheney

Most Frequently Asked Questions About NFTs(Non-Fungible Tokens)

Unknown · TechCrunch

Non-fungible tokens (NFTs) are the most popular digital assets today, capturing the attention of cryptocurrency investors, whales and people from around the world. People find it amazing that some users spend thousands or millions of dollars on a single NFT-based image of a monkey or other token, but you can simply take a screenshot for free. So here we share some frequently asked question about NFTs.

1) What is an NFT?

NFT stands for non-fungible token, which is a cryptographic token on a blockchain with unique identification codes that distinguish it from other tokens. NFTs are unique and not interchangeable, which means no two NFTs are the same. NFTs can be a unique artwork, GIF, Images, videos, Audio album. in-game items, collectibles etc.

2) What is Blockchain?

A blockchain is a distributed digital ledger that allows for the secure storage of data. By recording any kind of information—such as bank account transactions, the ownership of Non-Fungible Tokens (NFTs), or Decentralized Finance (DeFi) smart contracts—in one place, and distributing it to many different computers, blockchains ensure that data can't be manipulated without everyone in the system being aware.

3) What makes an NFT valuable?

The value of an NFT comes from its ability to be traded freely and securely on the blockchain, which is not possible with other current digital ownership solutions. The NFT points to its location on the blockchain, but doesn't necessarily contain the digital property. For example, if you replace one bitcoin with another, you will still have the same thing. If you buy a non-fungible item, such as a movie ticket, it is impossible to replace it with any other movie ticket because each ticket is unique to a specific time and place.

4) How do NFTs work?

One of the unique characteristics of non-fungible tokens (NFTs) is that they can be tokenised to create a digital certificate of ownership that can be bought, sold and traded on the blockchain.

As with crypto-currency, records of who owns what are stored on a ledger that is maintained by thousands of computers around the world. These records can't be forged because the whole system operates on an open-source network.

NFTs also contain smart contracts—small computer programs that run on the blockchain—that give the artist, for example, a cut of any future sale of the token.

5) What's the connection between NFTs and cryptocurrency?

Non-fungible tokens (NFTs) aren't cryptocurrencies, but they do use blockchain technology. Many NFTs are based on Ethereum, where the blockchain serves as a ledger for all the transactions related to said NFT and the properties it represents.5) How to make an NFT?

Anyone can create an NFT. All you need is a digital wallet, some ethereum tokens and a connection to an NFT marketplace where you'll be able to upload and sell your creations

6) How to validate the authenticity of an NFT?

When you purchase a stock in NFT, that purchase is recorded on the blockchain—the bitcoin ledger of transactions—and that entry acts as your proof of ownership.

7) How is an NFT valued? What are the most expensive NFTs?

The value of an NFT varies a lot based on the digital asset up for grabs. People use NFTs to trade and sell digital art, so when creating an NFT, you should consider the popularity of your digital artwork along with historical statistics.

In the year 2021, a digital artist called Pak created an artwork called The Merge. It was sold on the Nifty Gateway NFT market for \$91.8 million.

8) Can NFTs be used as an investment?

Non-fungible tokens can be used in investment opportunities. One can purchase an NFT and resell it at a profit. Certain NFT marketplaces let sellers of NFTs keep a percentage of the profits from sales of the assets they create.

9) Will NFTs be the future of art and collectibles?

Many people want to buy NFTs because it lets them support the arts and own something cool from their favorite musicians, brands, and celebrities. NFTs also give artists an opportunity to program in continual royalties if someone buys their work. Galleries see this as a way to reach new buyers interested in art.

10) How do we buy an NFTs?

There are many places to buy digital assets, like opensea and their policies vary. On top shot, for instance, you sign up for a waitlist that can be thousands of people long. When a digital asset goes on sale, you are occasionally chosen to purchase it.

11) Can i mint NFT for free?

To mint an NFT token, you must pay some amount of gas fee to process the transaction on the Ethereum blockchain, but you can mint your NFT on a different blockchain called Polygon to avoid paying gas fees. This option is available on OpenSea and this simply denotes that your NFT will only be

How we cleaned up and ranked our listeners' favorite albums of 2016

NPR Apps Blog

There's a new blog post describing the 2017 process here

All Songs Considered asks listeners for their favorite albums of 2016

At the beginning of December 2016, All Songs Considered followed a nice tradition and asked listeners for their favorite albums of 2016. Users could enter up to five different albums in a Google form, ranked according to their preferences. The poll was open for eight days and resulted in more than 4,500 entries.

In the end, the All Songs Considered team wanted a ranked list of the best albums. Sounds easy, right?

But data is always messy and there are a few problems to solve with this dataset. First, there were some obviously not-so-awesome things going on with the Google spreadsheet that gathers the results:

Different spelling, empty rows, multiple entries by one person: Ugh

In addition to cleaning up the data to make it usable, we had to decide on a weighting algorithm for the five different ranks and calculate it.

Since the whole project had a tight deadline, our process wasn't pretty, but we did it. Here's how:

Step 1: Combining Like Entries

The poll asks listeners to type in the artist and album, separated with a comma. But humans are faulty creatures who make spelling mistakes, don't obey the rules or don't remember the name of an album correctly. This faultiness results in a nice compilation of a dozen different ways to write one and the same thing:

Bon Iver - 22, a Million Bon Iver -22, A Million Bon Iver 22 a million Bon Iver, '22, A Million' BON IVER-22 A MILLION Bon Iver bon iver, "22, a million" Bon Iver, 22/10 Bon Iver, 20 a Million Bon Iver 22 bon iver, 22 a million Bon Iver, 22, A Million Bon Iver, 22a million BonIver, 22 a million Bon Iver, 33 a million Bon Iver, 22 million Bon Iver,22,a Million 22, A Million Bon Iver: 22, A million bon iver. 22,a million ...

...and that's still a relatively easy album name. I rely on your imagination to think of all the possible ways to spell "A Tribe Called Quest, We Got It from Here... Thank You 4 Your Service".

To fix that mess, we used a combination of cluster analysis in OpenRefine and "Find and Replace" in Google Spreadsheet.

First, OpenRefine. To run the cluster analysis on just one column instead of five different ones, we needed to trans-

form the data from a "wide" format into a "long" format. This can be easily achieved, e.g. with R:

```
library(reshape2) d = read.csv("data.csv",stringsAsFactors = FALSE) d = melt(d,id.vars = c("Timestamp")) write.csv(d,"data_long.csv")
```

Text Facet in OpenRefine

So what is cluster analysis? Basically, OpenRefine can run different algorithms on the data to cluster similar entries. Depending on the algorithm, "similar" is defined differently. OpenRefine offers different methods and keying functions, and we used all of them one after another.

Clustering in OpenRefine

OpenRefine then lets us select and merge similar entries and give them all a new name.

After successfully running through lots of different cluster methods, our data was approximately 95 percent clean. Our Bon Iver entries looked like this:

Bon Iver, 22, A Million Bon Iver, 22, A Million Bon Iver, 22, A Million Bon Iver, 22, A Million Bon Iver Bon Iver, 22, A Million Bon Iver, 22, A Million 22, A Million Bon Iver, 22, A Million Bon Iver, 22, A Million ...

So much better! But OpenRefine doesn't take care of the cases in which only the album or artist is mentioned. So we imported the data back into Google Spreadsheet and took care of that by hand – with a combination of "Find and Replace" and sorting the list alphabetically (which places all the Bon Iver's before Bon Iver, 22, A Million).

Step 2: Roughly clean up with a Python script

Once we made sure that the albums were written in the same way, they were countable. But we still needed to only count the entries that are from individual listeners who don't abuse the poll. To do so, we ran the cleaned data through a Python script. The Pandas library is a great choice for our first easy task, dropping the empty rows:

```
# Drop empty rows albums . dropna ( subset = RANKS )
```

But Pandas proved to be a bad choice for the next task: Deleting duplicate rows that appears within one hour. Doing that makes sure that we eliminated the entries that obviously come from one and the same person. We saw dozens of these copy-and-pasted entries (especially for the album Mind of Mine by Zayn). To get rid of all the duplicate entries within one hour, we first transformed the Pandas dataframe to a Python list and then checked for identical entries:

```
# Do row values match? If not, not a dupe for rank in RANKS : if row1 [ rank ] != row2 [ rank ]: return False
```

The last piece is checking for mentions of the same album within one entry. eg "Bevonce. Lemonade" on rank 1 and

Avoiding having to recompute values within SwiftUI views

Swift by Sundell

In general, using computed properties can be a great way to model view-specific data that we want to create on-demand, when need — especially within SwiftUI views, since each such view already uses a computed property (`body`) to implement its actual UI.

For example, here we're using two computed properties to determine what title and button text that a `UserSessionView` should render based on the app's current `LoginState` :

```
var body: some View { VStack { Text (title) . font (. headline ) .
foregroundColor ( . white ) Button (buttonText, action: but-
tonAction) . padding () . background ( Color . white ) .
cornerRadius ( 10 ) } . padding () . background ( Color .
blue ) . cornerRadius ( 15 ) }
```

```
private var title: String { switch state { case . loggedIn ( let
user): return "Welcome back, \(user.name) !" case . logged-
Out : return "Not logged in" } }
```

```
private var buttonText: String { switch state { case .
loggedIn : return "Log out" case . loggedOut : return "Log
in" } }
```

To learn more about the `Environment` property wrapper that's being used above, check out my guide to SwiftUI's state management system .

Implementing a SwiftUI view like that, using multiple computed properties, can often be a great approach — as it lets us keep our body implementations as simple as possible, and since it gives us a clear overview of how we're computing the different content that a given view will render.

Recomputed properties

But, we also have to keep in mind that computed properties are just that — computed — there's no form of caching or other kind of in-memory storage involved, meaning that each such values will always be recomputed each time that it's being accessed.

That's not an issue in our first example, because each of our properties can be quickly computed with constant (or $O(1)$) time complexity . However, let's now take a look at another example, which is quite different in terms of performance characteristics, since we're now computing a property by sorting a collection:

```
struct RemindersList: View { var items: [ Reminder . ID :
Reminder ]
```

```
var body: some View { List (sortedItems) { reminder in ... } }
```

```
private var sortedItems: [ Reminder ] { items.values . sorted
(by: { $0. dueDate Void
```

```
@State private var newItemName = ""
```

```
var body: some View { VStack { List (sortedItems) { reminder
in ... } HStack { TextField ( "Add a new reminder" ,
text: $newItemName ) Button ( "Add" ) { newItemHandler
( Reminder (name: newItemName)) } . disabled (newItem-
Name.isEmpty ) } . padding () } }
```

```
private var sortedItems: [ Reminder ] { items.values .
sorted (by: { $0. dueDate AnyPublisher { controller . load-
Reminders () } . map { items in items.values . sorted (by: { $0.
dueDate Void
```

```
init (items: [ Reminder . ID : Reminder ], newItemHandler:
@escaping ( Reminder ) -> Void ) { self . items = items.
values . sorted (by: { $0. dueDate < $1. dueDate }) self .
newItemHandler = newItemHandler }
```

```
@State private var newItemName = ""
```

```
... }
```

With the above change in place, we're now only sorting our collection once per `RemindersList` instance, rather than after every key stroke, without actually having to change the way our view is created or how our app's data is managed. So while my general recommendation is to keep initializers focused on simple setup work , rather than performing data mutations, that's a tradeoff that we might be willing to make in this case.

Worth keeping in mind, though, is that if the parent of our `RemindersList` view does update (or more specifically, if that parent's body property gets re-evaluated), then a new instance of our view is likely going to be created, meaning that we'll once again perform our item sorting operation.

Basically, when writing code within SwiftUI views, it's close to impossible to gain complete control over when and how that code will be executed. After all, a core part of the design of a declarative UI framework like SwiftUI is that the framework takes care of orchestrating all of our UI updates for us.

So if we wanted to improve our control over the lifecycle of our actual model logic, then a better approach will likely be to move that logic out from our view implementations and into objects that we have complete control over.

Dedicated model logic

One way to do that would be to use something like a view model to encapsulate the logic associated with handling our items array. If we then make that view model an `ObservableObject` , then we'll be able to easily observe it and connect to it within our SwiftUI views:

```
class RemindersListViewModel: ObservableObject { @Pub-
lished private(set) var items: [ Reminder ]
```

```
init (items: [ Reminder . ID : Reminder ]) { self . items =
items.values . sorted (by: { $0. dueDate < $1. dueDate }) }
```

```
func addItem(named name: String ) { }
```

What makes a great photo editing intern (Apply now for Summer 2019!)

NPR Apps Blog

This is not your standard photo internship!

This internship is an opportunity to learn more about the world of photo editing. Our goal isn't to make you into a photo editor; we view this internship as a chance for you to understand what it is like to be an editor and improve your visual literacy, which can help you become a better photographer.

Our paid internship runs from May 20 to Aug. 9. Applications are due Monday, March 4 at 5 p.m. ET.

What you will be doing

Editing: You'll be working closely with the Visuals Team's photo editors (Nicole and Emily) on fast-paced deadlines – we're talking anywhere from 15 minutes to publication, to short-term projects that are a week out. You'll dig into news coverage and photo research, learning how to communicate about what makes a good image across a range of news topics, including international, national, technology, arts and more.

Photography: Depending on the news cycle, there may be opportunities to photograph DC-area assignments. This can mean you'd have one or two shoots in a week, or maybe just a couple shoots in a month. You'll work closely with a radio or web reporter while out in the field, and a photo editor will go through your work and provide feedback for each assignment. There will also be a chance to work on portraiture and still lifes in our studio.

We also encourage each intern to create a self-directed project to work on throughout the semester. It can be an Instagram series, video, photo essay, text story or anything in-between. You can work independently or with another intern or reporter.

You will be part of NPR's intern program, which includes 40-50 interns each semester, across different departments. There will be coordinated training and intern-focused programming throughout the semester, which includes meeting NPR radio hosts, career development and other opportunities. As an intern, you will be treated as a member of the team. Many NPR employees are former interns and they're always willing to help current interns.

Any student (undergraduate or graduate), or person who has graduated no more than 12 months prior to the start of the internship period to which he/she is applying is eligible. Interns must be authorized to work in the United States.

Who should apply

We're looking for candidates that have a strong photojournalism background. An interest in editing, or experience with video/photo editing is a nice plus. It's also helpful if you've completed at least one photojournalism-focused in-

ternship prior to applying (let us know if you have!), though it's not necessary. A portfolio, however, is required.

We also want folks who can tell us what they would like to accomplish during their time at NPR. What do you want to learn? What do you want to try? We try to shape each internship around our intern, so we rely on you to tell us what goals you have for your time with us!

So how do I apply?

Does this sound like you? Read about our expectations and selection process and then apply now! . The application deadline has passed.

Into code, design, and data? Check out our design/development internship .

How to Setup the NPR App Template for You and Your News Org

NPR Apps Blog

On the NPR Visuals Team, we make a point to open source and publish as much of the code we write as we can. That includes open sourcing code like the app template, which we use every day to build the individual projects we make as a team.

However, we tend to optimize for ourselves rather than for the public, which means it can be a little more difficult for someone outside of our team to setup the app template. For this reason, I will walk through how to set up the app template for yourself if you are not a developer on our team. If you are unfamiliar with the app template, read more about it [here](#).

In this post, you will learn how to:

Ensure your development environment will work with the app template.

Set up a fork of the app template with your defaults.

Clone and bootstrap the app template for an individual project.

Deploy app template projects.

Customize the app template for your use and remove NPR branding.

Our app template relies on a UNIX-based development environment and working knowledge of the command line. We have a Python and Node-based stack. Thus, if you are new to all of this, you should probably read our development environment blog post first and make sure your environment matches ours. Namely, you should have Python 2.7 and the latest version of Node installed.

Also, all of our projects are deployed from the template to Amazon S3. You should have three buckets configured: one for production, one for staging and one for synchronizing large media assets (like images) across computers. For example, we use [apps.npr.org](#), [stage-apps.npr.org](#) and [assets.apps.npr.org](#) for our three buckets, respectively.

Cloning the template

All of our projects start and end in version control, so the first thing to do for your project is to fork our app template so you have a place for all of your defaults when you use the app template for more projects. This is going to be the place where all of your app template projects begin. When you want to start a new project, you clone your fork of the app template.

Once that is done, clone your fork to your local machine so we can start changing some defaults.

```
git clone git@github.com:$YOUR_GITHUB_USERNAME/
app-template.git
```

Set up your development environment

Hopefully, you've already checked to make sure your development stack matches ours. Next, we're going to create a virtual environment for the app template and install the Python and Node requirements. Use the following commands:

```
mkvirtualenv app-template pip install -r requirements.txt
npm install
```

Environment variables

You will also need a few environment variables established so that the entire stack works.

In order to use Google Spreadsheets with copytext from within the app template, you will need to store a Google username and password in your `bash_profile` (or comparable file for other shells like `zsh`).

```
export APPS_GOOGLE_EMAIL="youremail@gmail.com"
export APPS_GOOGLE_PASS="ih0pey0urpassw0rdisn0tpassword"
```

When you create spreadsheets for your projects, ensure the Google account stored in your environment can access the spreadsheet.

For deployment to Amazon S3, you will need your AWS Access Key ID and Secret stored as environment variables as well:

```
export AWS_ACCESS_KEY_ID="$AWSKEY" export
AWS_SECRET_ACCESS_KEY="$AWSSECRET"
```

After you have set these variables, open a new terminal session so that these variables are a part of your environment.

Setting your defaults

With your development environment and environment variables set, we can start hacking on the template.

All of the configuration you will need to change lives in `app_config.py`. Open that file in your text editor of choice. We will edit a few of the NPR-specific defaults in this file.

Change the following variables:

GITHUB_USERNAME : Change this to your (or your news org's) Github username.

PRODUCTION_S3_BUCKETS, **STAGING_S3_BUCKETS** and **ASSETS_S3_BUCKET** : You should change these dictionaries to the three buckets you have setup for this purpose. We also have a backup production bucket in case [apps.npr.org](#) goes down for any reason. Be sure to note the region of each S3 bucket.

Writing testable code when using SwiftUI

Swift by Sundell

A major part of the challenge of architecting UI-focused code bases tends to come down to deciding where to draw the line between the code that needs to interact with the platform's various UI frameworks, versus code that's completely within our own app's domain of logic.

That task might become especially tricky when working with SwiftUI, as so much of our UI-centric logic tends to wind up within our various View declarations, which in turn often makes such code really difficult to verify using unit tests.

So, in this article, let's take a look at how we could deal with that problem, and explore how to make UI-related logic fully testable — even when that logic is primarily used within SwiftUI-based views.

Logic intertwined with views

“You shouldn't put business logic within your views”, is a piece of advice that's often mentioned when discussing unit testing within the context of UI-based projects, such as iOS and Mac apps. However, in practice, that advice can sometimes be tricky to follow, as the most natural or intuitive place to put view-related logic is often within the views themselves.

As an example, let's say that we're working on an app that contains the following `SendMessageView`. Although the actual message sending logic (and its associated networking) has already been abstracted using a `MessageSender` protocol, all of the UI-specific logic related to sending messages is currently embedded right within our view:

```
struct SendMessageView: View { var sender: MessageSender
```

```
@State private var message = "" @State private var isSending = false @State private var sendingError: Error?
```

```
var body: some View { VStack { Text ("Your message:")
```

```
    TextEditor (text: $message)
```

```
    Button (isSending ? "Sending..." : "Send") { isSending = true sendingError = nil
```

```
Task { do { try await sender.sendMessage (message) message = "" } catch { sendingError = error }
```

```
isSending = false } } . disabled (isSending || message.isEmpty)
```

```
if let error = sendingError { Text (error.localizedDescription) . foregroundColor (.red) } } }
```

At first glance, the above might not look so bad. Our view isn't massive by any stretch of the imagination, and the code is quite well-organized. However, unit testing that view's logic would currently be incredibly difficult — as we'd have

to find some way to spin up our view within our tests, then find its various UI controls (such as its “Send” button), and then figure out a way to trigger and observe those views ourselves.

Because we have to remember that SwiftUI views aren't actual, concrete representations of the UI that we're drawing on-screen, which can then be controlled and inspected as we wish. Instead, they're ephemeral descriptions of what we want our various views to look like, which the system then renders and manages on our behalf.

So, although we could most likely find a way to unit test our SwiftUI views directly — ideally, we'll probably want to verify our logic in a much more controlled, isolated environment.

One way to create such an isolated environment would be to extract all of the logic that we're looking to test out from our views, and into objects and functions that are under our complete control — for example by using a view model. Here's what such a view model could end up looking like if we were to move all of our message sending UI logic out from our `SendMessageView`:

```
@MainActor class SendMessageViewModel: ObservableObject { @Published var message = "" @Published private(set) var errorText: String?
```

```
var buttonTitle: String { isSending ? "Sending..." : "Send" } var isSendingDisabled: Bool { isSending || message.isEmpty }
```

```
private let sender: MessageSender private var isSending = false
```

```
init (sender: MessageSender) { self.sender = sender }
```

```
func send() { guard !message.isEmpty else { return } guard !isSending else { return }
```

```
isSending = true errorText = nil
```

```
Task { do { try await sender.sendMessage (message) message = "" } catch { errorText = error.localizedDescription }
```

```
isSending = false } } }
```

To learn more about constructing and using observable objects, check out this guide to SwiftUI's state management system.

Our logic remains almost identical, but the above refactor does give us two quite significant benefits. First, we'll now be able to unit test our code without having to worry about SwiftUI at all. And second, we'll even be able to improve our SwiftUI view itself, as our view model now contains all of the logic that our view needs to decide how it should be rendered — making that UI code much simpler in the process:

```
struct SendMessageView: View { @ObservedObject var viewModel: SendMessageViewModel
```

TOP 11 AI MARKETING TOOLS YOU SHOULD USE (Updated 2022)

Unknown · TechCrunch

The marketing industry is turning to artificial intelligence (AI) as a way to save time and execute smarter, more personalized campaigns. 61% of marketers say AI software is the most important aspect of their data strategy.

If you're late to the AI party, don't worry. It's easier than you think to start leveraging artificial intelligence tools in your marketing strategy. Here are 11 AI marketing tools every marketer should start using today.

1. Jasper Ai (Formerly known as Jarvis)

Jasper is a content writing and content generation tool that uses artificial intelligence to identify the best words and sentences for your writing style and medium in the most efficient, quick, and accessible way.

It's trusted by 50,000+ marketers for creating engaging marketing campaigns, ad copy, blog posts, and articles within minutes which would traditionally take hours or days. Special Features:

Blog posts have been optimized for search engines and rank high on Google and other search engines. This is a huge plus for online businesses that want to generate traffic to their website through content marketing.

99.9% Original Content and guarantees that all content it generates will be original, so businesses can focus on their online reputation rather than worrying about penalties from Google for duplicate content.

Long-Form Article Writing – Jasper.ai is also useful for long-form writing, allowing users to create articles of up to 10,000 words without any difficulty. This is ideal for businesses that want to produce in-depth content that will capture their audience's attention.

Wait! I've got a pretty sweet deal for you. Sign up through the link below, and you'll get (10k Free Credits)

Also Read: Top 10 AI Content Generator & Writer Tools in 2022

2. Personalize

Personalize is an AI-powered technology that helps you identify and produce highly targeted sales and marketing campaigns by tracking the products and services your contacts are most interested in at any given time. The platform uses an algorithm to identify each contact's top three interests, which are updated in real-time based on recent site activity.

Key Features

Identifies top three interests based on metrics like time on page, recency, and frequency of each contact

Works with every ESP and CRM

Easy to get up and running in days

Enterprise-grade technology at a low cost for SMBs

3. Seventh Sense

Seventh Sense provides behavioral analytics that helps you win attention in your customers' overcrowded email inboxes. Choosing the best day and time to send an email is always a gamble. And while some days of the week generally get higher open rates than others, you'll never be able to nail down a time that's best for every customer. Seventh Sense eases your stress of having to figure out the perfect send-time and day for your email campaigns. The AI-based platform figures out the best timing and email frequency for each contact based on when they're opening emails. The tool is primarily geared toward HubSpot and Marketo customers

AI determines the best send-time and email frequency for each contact

Connects with HubSpot and Marketo

4. Phrasee

Phrasee uses artificial intelligence to help you write more effective subject lines. With its AI-based Natural Language Generation system, Phrasee uses data-driven insights to generate millions of natural-sounding copy variants that match your brand voice. The model is end-to-end, meaning when you feed the results back to Phrasee, the prediction model rebuilds so it can continuously learn from your audience.

Instantly generates millions of human-sounding, brand-compliant copy variants

Creates tailored language models for every customer

Learns what your audience responds to and rebuilds the prediction model every time

5. HubSpot Seo

HubSpot Search Engine Optimization (SEO) is an integral tool for the Human Content team. It uses machine learning to determine how search engines understand and categorize your content. HubSpot SEO helps you improve your search engine rankings and outrank your competitors. Search engines reward websites that organize their content around core subjects, or topic clusters. HubSpot SEO helps you discover and rank for the topics that matter to your business and customers.

Helps you discover and rank topics that people are searching for

Faster iOS performance debugging with ETTrace

Bruno Rocha · SwiftRocks

Performance problems can be rare in this era of blazing-fast iPhones, but chances are you're still having to deal with them every once in a while. Classic performance issues such as scroll hitches are still very easy to introduce if you're not careful about them, and when it happens, you'll want to be properly equipped to get to the bottom of it.

The "standard" way of debugging performance issues in iOS is to use Xcode's Time Profiler instrument, but I personally never had a good experience with it. While it contains all the information you need to understand a particular problem, that information is not exactly easy to make sense of. To make it worse, sometimes even getting the information to show up in the first place can be quite the challenge, as Instruments in iOS in general have been historically broken and plagued by bad UX.

Thankfully, you don't have to go through any of that! Today much better performance debugging tools are available (and for free), and in this article, I'll show you one of them.

Enter ETTrace

ETTrace is an open-source performance measurement framework for iOS developed by the folks behind Emerge , and I can say that today this is my favorite tool for measuring and debugging performance problems in iOS.

As mentioned in the beginning, while the Time Profiler does technically provide you with all the information that you need, actually understanding this information or even getting it to show up in the first place can be a big challenge, even if you know exactly what you're doing.

For me, personally, there are three things that make the Time Profiler hard to use. The first one is that you need to compile a special Profile build for it to work, meaning you cannot run it ad-hoc on an existing build or device. The second is that the Time Profiler has a really annoying tendency to simply refuse to work every once in a while, mostly when it comes to symbolication. Finally, last but not least, when you do manage to get it to work, the way in which the data is presented to you is not very helpful when it comes to locating the source of a particular performance bottleneck. In other words, there are better ways to display this data.

ETTrace, on the other hand, has none of these problems. It doesn't require a special build, it automatically handles symbolication for you, and it displays the data in a much more readable way. It's basically the Time Profiler on steroids, and I have found it to be in most cases a complete replacement for it.

Example: Using ETTrace to find and fix a bottleneck

For instructions on how to install ETTrace, check out the official repo. As of writing, ETTrace is installed by linking a dynamic framework into your app and installing a special

ettrace CLI tool in your Mac. You can trace any build of your app that links against this framework, which is why you don't need to compile a special Profile build like you would when using Xcode and the Time Profiler. In practice you could even ship this framework alongside your App Store builds in order to be able to directly debug issues found in production, but I would personally not do that and keep it restricted to debug builds.

To see how ETTrace can help us debug performance issues better than the standard Time Profiler, let's pretend that we have a view controller called `ExploreCardViewController` , and that we have noticed that tapping a specific collection view cell in this VC is causing the app to freeze for a while.

To find out exactly why this is happening, we just need to run ETTrace. After following the usage steps as described on the repo, you'd be presented with something like this:

This way of displaying information is called a Flame Graph , and I find it to be a very efficient way of locating performance bottlenecks in your app's code. Each "entry" that you see here is a single method call in your app, with the X axis dictating when it was called (and how long it took to run), and the Y axis dictating where/who called it. In the example above, the first 3 frames (start/main/UIApplicationMain) represent functions internal to iOS that are responsible for launching and keeping the app alive, while everything else below it is actual code from our example app.

To find performance bottlenecks in a flame graph, all we need to do is look for the presence of a "chunky" stack trace and then go down the Y axis until we find which frame exactly is the source of the chunkiness.

Consider how `ExploreCardViewController` is shown in the report. It's very large, which means that this method is taking a really long time to run. But what exactly is causing it? Is it the literal call to `didSelectItemAt` , or is it something else further down the stack trace?

By going down the trace we can see that at its very bottom there's a very expensive call to `usleep` originating from `ArticleViewController.viewDidLoad()` , which is the reason why that entire stack trace is being reported as being expensive:

Oops, seems like we forgot some debug code in our class!

```
func viewDidLoad() { sleep(1) /\ TODO: remove this! }
```

After deleting the call, the bottleneck was gone!

You may find this to be a dumb example, but I find that debugging real performance issues doesn't stray too far from this. The difference is just that instead of a dumb call to `sleep` , you'd see some other expensive operation. Otherwise, the process to locate it and the different ways in which you could fix it are the same.

Observing the content offset of a SwiftUI ScrollView

Swift by Sundell

When building various kinds of scrollable UIs, it's very common to want to observe the current scroll position (or content offset, as UIScrollView calls it) in order to trigger layout changes, load additional data when needed, or to perform other kinds of actions depending on what content that the user is currently viewing.

However, when it comes to SwiftUI's ScrollView, there's currently (at the time of writing) no built-in way to perform such scrolling observations. While embedding a ScrollViewReader within a scroll view does enable us to change the scroll position in code, it strangely (especially given its name) doesn't let us read the current content offset in any way.

One way to solve that problem would be to utilize the rich capabilities of UIKit's UIScrollView, which — thanks to its delegate protocol and the scrollViewDidScroll method — provides an easy way to get notified whenever any kind of scrolling occurred. However, even though I'm normally a big fan of using UIViewRepresentable and the other SwiftUI/UIKit interoperability mechanisms, in this case, we'd have to write quite a bit of extra code to bridge the gap between the two frameworks.

That's mainly because — at least on iOS — we can only embed SwiftUI content within a UIHostingController, not within a self-managed UIView. So if we wanted to build a custom, observable version of ScrollView using UIScrollView, then we'd have to wrap that implementation in a view controller, and then manage the relationship between our UIHostingController and things like the keyboard, the scroll view's content size, safe area insets, and so on. Not impossibly by any means, but still, a fair bit of additional work and complexity.

So, let's instead see if we can find a completely SwiftUI-native way to perform such content offset observations.

Resolving frames using GeometryReader

One thing that's key to realize before we begin is that both UIScrollView and SwiftUI's ScrollView perform their scrolling by offsetting a container that's hosting our actual scrollable content. They then clip that container to their bounds to produce the illusion of the viewport moving. So if we can find a way to observe the frame of that container, then we'll essentially have found a way to observe the scroll view's content offset.

That's where our good old friend GeometryReader comes in (wouldn't be a proper SwiftUI layout workaround without it, right?). While GeometryReader is mostly used to access the size of the view that it's hosted in (or, more accurately, that view's proposed size), it also has another neat trick up its sleeve — in that it can be asked to read the frame of the current view relative to a given coordinate system.

To use that capability, let's start by creating a PositionObservingView, which lets us bind a CGPoint value to the current position of that view relative to a CoordinateSpace that we'll also pass in as an argument. Our new view will then embed a GeometryReader as a background (which will make that geometry reader take on the same size as the view itself) and will assign the resolved frame's origin as our offset using a preference key — like this:

```
var body: some View { content() . background ( GeometryReader { geometry in Color . clear . preference ( key: PreferenceKey . self , value: geometry . frame ( in: coordinateSpace ). origin ) }) . onPreferenceChange ( PreferenceKey . self ) { position in self . position = position } } }
```

To learn more about how the @ViewBuilder attribute can be used when building custom SwiftUI container views, check out this article.

The reason we use SwiftUI's preference system above is because our GeometryReader will be invoked as part of the view updating process, and we're not allowed to directly mutate our view's state during that process. So, by using a preference instead, we can deliver our CGPoint values to our view in an asynchronous fashion, which then lets us assign those values to our position binding.

Now all that we need to do is to implement the PreferenceKey type that's used above, and we'll be good to go:

```
private extension PositionObservingView { struct PreferenceKey: SwiftUI . PreferenceKey { static var defaultValue: CGPoint { . zero } }
```

We don't actually need to implement any kind of reduce algorithm above, since we'll only have a single view delivering values using that preference key within any given hierarchy (since our implementation is entirely contained within our PositionObservingView).

Alright, so now we have a view that's capable of reading and observing its own position within a given coordinate system. Let's now use that view to build a ScrollView wrapper that'll let us accomplish our original goal — to be able to read the current content offset within such a scroll view.

From position to content offset

Our new ScrollView wrapper will essentially have two responsibilities — one, it'll need to convert the position of our inner PositionObservingView into the current scroll position (or content offset), and two, it'll also need to define a CoordinateSpace that the inner view can use to resolve its position. Besides that, it'll simply forward its configuration parameters to its underlying ScrollView, so that we can decide what axes we want each scroll view to operate on, and so that we can decide whether or not to display any scrolling indicators.

The good news is that converting our inner view's position into content offset is as easy as negating both the x and

Mutating and non-mutating Swift contexts

Swift by Sundell

One of the ways in which Swift helps us write more robust code is through its concept of value types, which limit the way that state can be shared across API boundaries. That's because, when using value types, all mutations are (by default) only performed to local copies of the values that we're working with, and APIs that actually perform mutations have to be clearly marked as mutating.

In this article, let's explore that keyword, as well as its nonmutating counterpart, and the sort of capabilities that those language features provide.

What can a mutating function do?

Essentially, a function that's been marked as mutating can change any property within its enclosing value. The word "value" is really key here, since Swift's concept of structured mutations only applies to value types, not to reference types like classes and actors.

For example, the following Meeting type's cancel method is mutating, since it modifies its enclosing type's state and reminderDate properties:

```
struct Meeting { var name: String var state: MeetingState var reminderDate: Date ? ...
```

```
mutating func cancel(withMessage message: String) { state = . cancelled (message: message) reminderDate = nil } }
```

Besides modifying properties, mutating contexts can also assign a brand new value to self, which can be really useful when adding a mutating method to an enum (which can't contain any stored instance properties). For example, here we're creating an API for making it easy to appending one Operation to another:

```
enum Operation { case add( Item ) case remove( Item ) case update( Item ) case group([ Operation ] ) }
```

```
extension Operation { mutating func append( _ operation: Operation ) { self = . group ( [ self , operation] ) } }
```

The above technique also works for other value types, such as structs, which can be really useful if we ever want to reset a value back to its default set of properties, or if we want to mutate a more complex value as a whole — for example like this:

```
struct Canvas { var backgroundColor: Color ? var foregroundColor: Color ? var shapes = [ Shape ]() var images = [ Image ]()
```

```
mutating func reset() { self = Canvas () } }
```

The fact that we can assign a brand new value to self within a mutating function might initially seem a bit strange, but we have to remember that Swift structs are really just values

— so just like how we can replace an Int value by assigning a new number to it, we can do the same thing with any other struct (or enum) as well.

Mutating protocol requirements

Although the concept of separating mutating and non-mutating APIs is something that's unique to value types, we can still make a mutating function a part of a protocol as well — even if that protocol might end up being adopted by a reference type, such as a class. Classes can simply omit the mutating keyword when conforming to such a protocol, since they are inherently mutable.

What's very interesting, though, is that if we extend a protocol with a default implementation of a mutating function, then we could implement things like the above reset API without actually knowing what type of value that we're resetting — like this:

```
protocol Resettable { init () mutating func reset() }
```

```
extension Resettable { mutating func reset() { self = Self () } }
```

```
struct Canvas: Resettable { var backgroundColor: Color ? var foregroundColor: Color ? var shapes = [ Shape ]() var images = [ Image ]() }
```

Performing mutations within initializers

While functions always need to be explicitly marked as mutating whenever we want to modify a value type's internal state (whether that's a property, or the entire value itself), initializers are always mutating by default. That means that, besides assigning initial values to a type's properties, an initializer can also call mutating methods to perform its work (as long as self has been fully initialized beforehand).

For example, the following ProductGroup calls its own add method in order to add all of the products that were passed into its initializer — which makes it possible for us to use a single code path for that logic, regardless of whether it's being run as part of the initialization process or not:

```
struct ProductGroup { var name: String private(set) var products = [ Product ]() private(set) var totalPrice = 0
```

```
init (name: String , products: [ Product ] ) { self . name = name products . forEach { add ($0) } }
```

```
mutating func add( _ product: Product ) { products . append (product) totalPrice += product . price } }
```

Just like mutating functions, initializers can also assign a value directly to self. Check out this quick tip for an example of that.

Non-mutating properties

So far, all of the examples that we've been taking a look at have been about mutable contexts, but Swift also offers a way to mark certain contexts as explicitly non-mutating as

How we approached data cleaning for our listeners' favorite albums of 2017

NPR Apps Blog

All Songs Considered asks listeners for their favorite albums of 2017

As 2017 was coming to an end, All Songs Considered continued a longstanding tradition and asked listeners for their favorite albums of 2017. Users could enter up to five different albums and artist pairs in a Google Form, ranked according to their preferences. The poll was open for eight days and resulted in more than 4,800 entries.

In the end, the All Songs Considered team wanted a ranked list of the best albums. We already dove into this task in 2016, and you can check the process in this old post. However, after reviewing code snippets from last year we decided to take a new approach this year. In this post we will explain the decisions we made and why we think it is an improvement that will allow us to move forward more quickly in coming years.

Our main goals this year were:

Improve the submission form to start with a cleaner dataset.

Use a more programmatic approach to cluster similar entries for artist/album pairs.

Use make to tie together our data processing pipeline. We were inspired by the awesome explanations of the use of Makefiles for data processing by Datamade.

Reuse ranking strategy from the previous year.

Let's dive into each of those goals and how we tackled them.

Form improvements

Our first question was "is there a way to improve the audience submission Google form to provide cleaner initial results?" In previous years we had asked our audience to add a comma-separated artist and album pair, but that rule was not always followed. Surprise! Welcome to the world of free text fields.

This year we made two major changes. First, we decided to split album and artist into separate inputs grouped under a common heading. Second, we made the #1 album and artist fields required.

2017 form

Looking back to that decision, it made a huge difference in the cleanliness of the original dataset. The separation of album and artist gave us a consistent data format and the required fields eliminated completely empty rows. It was useful to have a better starting point for our cleanup process.

Original export of the 2017 form

Clustering similar artist/album entries

Members of our team had already tried out dedupe in other projects and we thought this data cleaning task was well suited for that tool. We want to identify and group together similar artist/album pairs in order to correctly rank them later.

We finally used `csvdedupe`, a command line tool for deduplicating CSV files, that takes a messy input file or data piped from standard input and identifies duplicates.

Installation is as easy as using `pip`:

```
pip install csvdedupe
```

Preparing data for `csvdedupe`

In order to use `csvdedupe` effectively, we first needed to change the format of the input data so that every artist/album pair was in a different row. That way `csvdedupe` could work across all of the entries and group them.

One artist/album pair per row

As you can see in the above CSV output, we have assigned points to each album/artist pair depending on the position in the Google form. We wanted a point schema that would always rank two votes for an artist/album pair higher than a single vote, regardless of the single vote's position.

So in order to comply with the above rule, we have assigned points like this:

15 points to album/artist #1

14 points to album/artist #2

13 points to album/artist #3

12 points to album/artist #4

11 points to album/artist #5

Training `csvdedupe`

`Dedupe` uses a supervised machine learning algorithm to detect what we want to identify as similar. The first step in using it is training the algorithm. `Dedupe` suggests that we provide at least 10 positive results (similar entries) and 10 negative results (dissimilar entries) for it to build a model that can give us accurate results.

Training `dedupe` on the terminal

Verifying `csvdedupe` results

`Dedupe` was a huge improvement in terms of simplifying our detection of similar entries, but we wanted some kind of manual verification step to fine tune entries that were either bundled together incorrectly or not bundled together when

Swift by Sundell turns five years old today! Here's what's next for the website and the podcast

Swift by Sundell

Today it's been exactly five years since this website first launched, and what a journey it has been! When I started this website after having published weekly articles on Medium for a few months, I honestly had no idea what to expect. I wasn't even sure if anyone would even find the site to begin with, let alone be interested in coming back week after week to read my new articles.

But now, five years later, and with over 550 articles and 115 podcast episodes published, this website has been visited by over a million Swift developers from all around the world, and hundreds of thousands of people come back to read new articles and listen to new podcast episodes every single month. To see this website grow like that has been absolutely incredible, and I'm eternally grateful for all of the amazing support that you — the Swift community — have given me over these first five years.

During these first few years, the amount of time that I've been able to spend working on my articles and podcast episodes has varied quite a lot. In the beginning, I was simply working on Swift by Sundell during my spare time, as a hobby project, and I had no plans or intentions to ever scale the project much beyond that. But at the same time, the website kept growing, and so did my ambitions, and at the start of 2019 (two years after the initial launch) I made the rather bold decision to turn Swift by Sundell into my full-time job.

In retrospect, I'm incredibly happy that I made that decision, not only because it enabled me to produce a much larger number of more detailed articles, and spend more time on podcast editing to make each episode sound great — but also because it led to the creation of Ink, Plot, and Publish, and the current iteration of this website which uses those tools to produce a fast, easy to use, statically generated site that I'm super proud of.

Working full-time on your own projects, as amazing as it in many ways is, also comes with quite a few downsides, though. In particular, I really started to miss working with other people, to be a part of a team, and to do actual iOS and macOS app development. Building Publish and my other open source tools was a lot of fun, and involved solving a ton of interesting problems, but it just wasn't the same as working on complex app projects with dedicated co-workers.

So in 2020 I decided to start freelancing again, part-time, while still keeping Swift by Sundell as my main, full-time job — which worked out great. In fact, it worked out so great that I started taking on more freelancing work throughout 2021, and for a while, I was incredibly happy with how my work was progressing. Business was great, with income from both this website's sponsorships and from my freelancing work, I got to work on some incredibly cool projects, plus those projects gave me such a huge amount of inspiration

for both my articles and podcast episodes. It all turned out to be one big, positive feedback loop.

However, towards the end of 2021, I started to feel that something was wrong. I very often felt incredibly stressed, and sometimes even deeply disappointed in myself for not writing enough articles, for not publishing podcast episodes frequently enough, for not reviewing open source pull requests fast enough, or for not working enough hours on a given project.

Looking back, it's quite obvious what had happened. As I kept constantly expanding all of my projects and client work over time, I had ended up with more or less two full-time jobs, and it became clear that I was at the edge (or, some may say, even past the edge) of being burned out. Something had to change.

So, at the beginning of this year, I sat down and started making a plan. I knew that I didn't want to stop doing client work (I still work on some really nice projects with incredible teams), and I also knew that I didn't want to quit working on this website, the podcast, Stacktrace (my other podcast), or my open source projects. So I just needed to come up with some kind of compromise, a schedule that would let me keep working on all of those things that I'm super passionate about, while also making that schedule much more sustainable and healthy.

What I've come up with will likely disappoint some of you, since it does involve a reduction of the amount of time that I'll be able to spend on Swift by Sundell going forward (and the number of articles and podcast episodes that I'll be able to make as a result), but I want to make it crystal clear that I'm making these changes because I want to keep working on Swift by Sundell for many more years to come. I love this website and the wonderful community that exists around Swift and Apple's platforms, and I want to continue contributing to that community as much as I possibly can far into the future.

So, what changes am I making? First, I'm removing all sorts of publishing schedules, and will instead post new articles and podcast episodes on a more infrequent basis. While I'm aiming to publish at least one new article and one podcast episode per month, there might occasionally be some periods of time where not much is happening on this website, but that doesn't mean that I've abandoned it, I might just be busy with other work. I've always prioritized quality over quantity, and I'd much rather take my time writing a really great article than rush one out just to meet a specific schedule.

Next, I'm removing all ads from the website, starting next week, on the 9th of May. That's right, you'll soon be able to enjoy all of this website's hundreds of articles and all of my Discover guides completely ad-free, all free of charge. While I might occasionally publish a (clearly marked) sponsored post, and there will still be sponsorships on the podcast, I want to turn my articles back to what they started

Multivariate testing: Learning what works from your users at scale

NPR Apps Blog

Multivariate and AB testing are generally used to iterate on products over time. But what do you do when your product is always different, like the visual stories we tell?

For the past year, NPR Visuals has been iterating on a story format for picture stories that works like a slideshow, presenting full-width cards with photos, text and any other HTML elements. We have made various tweaks to the presentation, but since each story is substantially different, it's hard to know definitively what works.

With three stories approaching launch in the middle of February ("A Brother And Sister In Love" , "Life After Death" and "A Photo I Love: Thomas Allen Harris"), we decided to test different ways to get a user to take action at the end of a story. We wanted to encourage users to support NPR or, in the case of "A Brother And Sister In Love" and "A Photo I Love", to follow our new project Look At This on social media.

To find out, we conducted live experiments using multivariate testing, a research method that allows us to show users slightly different versions of the same page and assess which version people respond to more positively.

In multivariate testing, you determine a control scenario (something you already know) and form a hypothesis that a variation of that scenario would perform better than the control.

(Note : You will see the term multivariate testing, A/B testing or split testing to discuss experiments like this. While there is a technical difference between the implementation of these various methods, they all seek to accomplish the same thing so we are not going to worry too much about accuracy of the label for the purposes of discussing what we learned.)

We call this question, which changed slightly on each project, the "Care Question", as it always tried to gauge whether a user cared about a story.

The overall test model worked like this:

The test exposed two possible paths to users

When we ran the test, we showed half of users (who reached the final slide) the Care Question with two buttons, "Yes" and "No". Clicking Yes brought them to one of the two actions listed above, clicking No revealed a prompt to email us feedback. The control group was shown the action we wanted them to take, without a preceeding question.

We were able to run these tests at about equal intervals with a small amount of code .

In this blog post, we will show the results, how we determined them and what we learned.

When a user reached the conclusion slide, we sent an event to Google Analytics to log which set of tests ran.

We also tracked clicks on the "Yes" and "No" buttons of the Care Question, and clicks on the subsequent actions (support link, each of the follow links, and the feedback email link).

The Care Question used in A Brother And Sister In Love

Determining whether the results were statistically significant required some pretty complex calculations, which you can read about here . Luckily, Hubspot provides a simple-to-use calculator to determine the statistical significance of your results.

Significance is determined by the confidence interval, or how confident you can be that your numbers are not determined simply by randomness. Usually, a 95% confidence interval or greater is high enough to draw a conclusion. Using the calculator, we determined whether the difference in conversion rates (where conversion rate is defined as clicks over the number of times a particular test was run) was statistically significant.

"A Brother And Sister In Love"

The test for "A Brother And Sister In Love" was actually two separate A/B tests at the same time: whether a user was prompted to follow Look At This on social media or support NPR by donating. For each scenario, users were prompted with the Care Question or not. The Care Question was "Did you love this story?"

This breaks down into two tests, a "follow test" and a "support test", with a control and variation scenario for each:

Follow test, control

Follow test, variation

Support test, control

Support test, variation

Finally, if a user clicked no, we provided a prompt to email us feedback.

If a user answered "No" to the Care Question, we asked them to email us feedback.

We were able to determine with 99.90% confidence that prompting a user with a question before asking them to "Support Public Radio" was more successful. We converted 0.184% of users who did not receive the Care Question and 1.913% of users who did, which makes a user who received the Care Question 10 times more likely to click the support link.

"Life After Death"

What makes a great photo editing intern (Apply now for summer 2018!)

NPR Apps Blog

This is not your standard photo internship!

This internship is an opportunity to learn more about the world of photo editing. Our goal isn't to make you into a photo editor; we view this internship as a chance for you to understand what it is like to be an editor and improve your visual literacy, which can help you become a better photographer.

The paid internship runs from May 21, 2018 to August 10, 2018. Applications are due Sunday, March 4th at 11:59pm eastern .

What you will be doing

Editing: You'll be working closely with the Visuals Team's photo editors (Nicole and Emily) on fast-paced deadlines – we're talking anywhere from 15 minutes to publication, to short-term projects that are a week out. You'll dig into news coverage and photo research, learning how to communicate about what makes a good image across a range of news topics, including international, national, technology, arts and more.

Photography: Depending on the news cycle, there may be opportunities to photograph DC-area assignments. This can mean you'd have one or two shoots in a week, or maybe just a couple shoots in a month. You'll work closely with a radio or web reporter while out in the field, and a photo editor will go through your work and provide feedback for each assignment. There will also be a chance to work on portraiture and still lifes in our studio.

We also encourage each intern to create a self-directed project to work on throughout the semester. It can be an Instagram series, video, photo essay, text story or anything in-between. You can work independently or with another intern or reporter .

You will be part of NPR's intern program, which includes 40-50 interns each semester, across different departments. There will be coordinated training and intern-focused programming throughout the semester, which includes meeting NPR radio hosts, career development and other opportunities. As an intern, you will be treated as a member of the team. Many NPR employees are former interns and they're always willing to help current interns.

Any student (undergraduate or graduate), or person who has graduated no more than 12 months prior to the start of the internship period to which he/she is applying is eligible. Interns must be authorized to work in the United States.

Who should apply

We're looking for candidates that have a strong photojournalism background. An interest in editing, or experience with video/photo editing is a nice plus. It's also helpful if

you've completed at least one photojournalism-focused internship prior to applying (let us know if you have!), though it's not necessary. A portfolio, however, is required.

We also want folks who can tell us what they would like to accomplish during their time at NPR. What do you want to learn? What do you want to try? We try to shape each internship around our intern, so we rely on you to tell us what goals you have for your time with us!

So how do I apply?

Does this sound like you? Read about our expectations and selection process and then apply now!

Into code, design, and data? Check out our design/development internship .

There will be coordinated training and intern-focused programming throughout the semester, which includes meeting NPR radio hosts, career development and other opportunities.

NPR Apps Blog

Combining protocols in Swift

Swift by Sundell

One of the core strengths of Swift's protocols is that they enable us to define shared interfaces that multiple types can conform to, which in turn lets us interact with those types in a very uniform way, without necessarily knowing what underlying type that we're currently dealing with.

For example, to clearly define an API that enables us to persist a given instance onto disk, we might choose to use a protocol that looks something like this:

```
protocol DiskWritable { func writeToDisk(at url: URL )
throws }
```

One advantage of defining commonly used APIs that way is that it helps us keep our code consistent, as we can now make any type that should be disk-writable conform to the above protocol, which then requires us to implement the exact same method for all such types.

Another big advantage of Swift protocols is that they're extendable, which makes it possible for us to define all sorts of convenience APIs for both our own protocols, as well as those that are defined externally — for example within the standard library, or within any framework that we've imported.

When writing those kinds of convenience APIs, we might also want to mix the protocol that we're currently extending with some functionality provided by another protocol. For example, let's say that we wanted to provide a default implementation of our DiskWritable protocol's writeToDisk method for types that also conform to the Encodable protocol — since a type that's encodable can be transformed into Data, which we could then automatically write to disk.

One way to make that happen would be to make our DiskWritable protocol inherit from Encodable, which in turn will require all conforming types to implement both of those two protocols' requirements. We could then simply extend DiskWritable in order to add that default implementation of writeToDisk that we were looking to provide:

```
protocol DiskWritable: Encodable { func writeToDisk(at url:
URL ) throws }
```

```
extension DiskWritable { func writeToDisk(at url: URL )
throws { let encoder = JSONEncoder () let data = try encoder.
encode ( self ) try data. write (to: url) } }
```

While powerful, the above approach does have a quite significant downside, in that we've now completely coupled our DiskWritable protocol with Encodable — meaning that we can no longer use that protocol by itself, without also requiring any conforming type to also fully implement Encodable, which might become problematic.

Another, much more flexible approach would be to let DiskWritable remain a completely stand-alone protocol,

and instead write a type-constrained extension that only adds our default writeToDisk implementation to types that also conform to Encodable separately — like this:

```
extension DiskWritable where Self : Encodable { func write-
ToDisk(at url: URL ) throws { let encoder = JSONEncoder
() let data = try encoder. encode ( self ) try data. write (to:
url) } }
```

The tradeoff here is that the above approach does require each type that wants to leverage our default writeToDisk implementation to explicitly conform to both DiskWritable and Encodable, which might not be a big deal, but it could make it a bit harder to discover that default implementation — since it's no longer automatically available on all DiskWritable-conforming types.

One way to address that discoverability issue, though, could be to create a convenience type alias (using Swift's protocol composition operator, &) that gives us an indication that DiskWritable and Encodable can be combined to unlock new functionality:

```
typealias DiskWritableByEncoding = DiskWritable & Encod-
able
```

```
} class EditorViewController: UIView-
Controller { private var document:
Document private var fileURL: URL.
```

Swift by Sundell

When a type conforms to those two protocols (either using the above type alias, or completely separately), it'll now get access to our default writeToDisk implementation (while still having the option to provide its own, custom implementation as well):

```
struct TodoList: DiskWritableByEncoding { var name: String
var items: [ Item ] ... }
```

```
let list = TodoList (...) try list. writeToDisk (at: fileURL)
```

Combining protocols like that can be a really powerful technique, as we're not just limited to adding default implementations of protocol requirements — we can also add brand new APIs to any protocol combination, simply by adding new methods or computed properties within one of our extensions.

For example, here we've added a second overload of our writeToDisk method, which makes it possible to pass a custom JSONEncoder that'll be used when serializing the current instance:

```
extension DiskWritable where Self : Encodable { func write-
ToDisk(at url: URL , encoder: JSONEncoder ) throws { let
data = try encoder. encode ( self ) try data. write (to: url) }
```

```
func writeToDisk(at url: URL ) throws { try writeToDisk (at:
url, encoder: JSONEncoder ()) } }
```

Welcome to the new Project Zero Blog

Natalie Silvanovich · Project Zero (Google)

While on Project Zero, we aim for our research to be leading-edge, our blog design was ... not so much. We welcome readers to our shiny new blog!

For the occasion, we asked members of Project Zero to dust off old blog posts that never quite saw the light of day. And while we wish we could say the techniques they cover are no longer relevant, there is still a lot of work that needs to be done to protect users against zero days. Our new blog will continue to shine a light on the capabilities of attackers and the many opportunities that exist to protect against them.

From 2016: Windows Exploitation Techniques: Race conditions with path lookups by James Forshaw

Podcast: “The magic of Augmented Reality”, with special guest Roxana Julia

Swift by Sundell

Roxana Julia joins John to discuss Apple’s various tools, frameworks, and APIs for building Augmented Reality experiences. How to get started building AR-based apps, what’s the current state of AR on Apple’s platforms, and how might that change if Apple were to introduce dedicated AR hardware in the future?

Stream : Build scalable in-app chat or activity feeds in record time using Stream’s open source, Swift-native SDK that’s backed by enterprise-grade server infrastructure. Get started for free at getstream.io/sundell , or browse their open job positions at getstream.io/team .

Instabug : Incredibly useful bug, crash, and performance monitoring for iOS apps. Go to instabug.com/sundell to start your free trial.

[Roxana on Twitter](#)

[John on Twitter](#)

[Roxana’s website](#)

[ARKit](#)

[RealityKit](#)

[SceneKit](#)

[SpriteKit](#)

[Reality Converter and Reality Composer](#)

[The USDZ file format](#)

[Child view controllers](#)

[Max Frazer’s RealityKit shader tutorial](#)

[Ryan Kopinsky’s “Reality School” YouTube channel](#)

[Apple’s Human Interface Guidelines for AR](#)

[ARCoachingOverlayView](#)

[LiDAR](#)

[ARDepthData](#)

[IKEA Place](#)

[The “Create 3D models with Object Capture” WWDC session](#)

[SwiftUI](#)

Microblog: TestMain can cause one to question reality

Dave Cheney

This morning a one line change had several of us tearing up the fabric of reality trying to understand why a failing test wasn't failing, or, in fact, being run at all. Increasingly frantic efforts to upgrade/downgrade Go, run the tests on another machine, run the tests in CI, all served to only unnerve us further.

Can you spot the bug?

```
package gosh_darn_important_test
```

```
import ( "testing" "go.uber.org/goleak" )
```

```
func TestMain(m *testing. M) { /\ goleak. VerifyTest-  
Main(m) }
```

```
...
```

TestMain is intended to be a middleware (so to speak) between the go test test runner and the Test functions in a package. The goleak check had recently been commented out (not surprisingly because of yet another logging library that spawns goroutines in the background) which meant that TestMain no longer called, indirectly, m. Run to execute any tests.

Scripting News

Andy Baio noted that it was 20 years ago today that Jack Dorsey posted his first tweet . He also noted it was the day that Ze Frank did his first YouTube video. It got me looking around my own world to see what happened on Mar 21, 2006 . Nothing earth shaking but it was interesting piece written by Phil Jones on how everyone watched me all the time and they were all trying to figure out what I do. Fact: At the time I was trying to make OPML grow big like RSS had, but it didn't happen. The big concept was the World Outline that would be an open directory where everyone created browsable outlines that linked to their own outlines and others, in a completely fluid way. In order to be something it had to catch on, and it didn't.

In the intervening twenty years, I tried it again and again to start a technology party like blogging and podcasting, viral viralities – but nothing stuck. I came close once, with Twittergram , but I didn't want to run a company, I wanted to keep developing software. Sold it to Betaworks , but they never marketed it. Instead I helped them launch bit.ly and had a blast doing that. I love doing PR. Anyway I guess I got lazy. And I wasn't building on the web any longer. Instead I was trying to fit in between Twitter and Facebook mostly. Now I'm getting ready, much older and more tired, but wiser – to go back to roots, to use WordPress as my blogging platform, as if it were Frontier – and see what we can build out of the web and if it'll stick. That's why I'm so

relentless at getting people to play with me.

It's the same damn thing Phil Jones describes. And OPML is going to be a big part, yet again – only this time playing a vastly different role, with lists of feed locations on the web. If it works we will call it the feediverse . Even if it doesn't work. ;-)

Sponsor: ViRE

Swift by Sundell

Thanks a lot to the team behind the visual regular expressions editor ViRE for sponsoring Swift by Sundell over the past two weeks, enabling me to keep delivering a continuous stream of Swift articles completely for free.

I'll be honest, I've never been a huge fan of working with regular expressions as raw strings. I find them hard to read, hard to debug, and the syntax to be quite cryptic. That being said, there are still situations in which using regular expressions is a great choice, especially if you want to parse or validate strings that conform to a, well, regular format.

That's why ViRE is such a fantastic app. Not only does it give you a visual editor that makes it so much easier to work with all sorts of regular expressions, it also includes a ton of really useful features — such as a unit testing tool (yes, you can now unit test your regular expressions, how cool isn't that?), great documentation, and much more. Plus, the app is fast, well-designed, and is available on both the Mac and the iPad.

→ Try it for free today, and support Swift by Sundell in the process

Use internal packages to reduce your public API surface

Dave Cheney

In the beginning, before the go tool, before Go 1.0, the Go distribution stored the standard library in a subdirectory called `pkg/` and the commands which built upon it in `cmd/`. This wasn't so much a deliberate taxonomy but a by product of the original make based build system. In September 2014, the Go distribution dropped the `pkg/` subdirectory, but then this tribal knowledge had set root in large Go projects and continues to this day.

I tend to view empty directories inside a Go project with suspicion. Often they are a hint that the module's author may be trying to create a taxonomy of packages rather than ensuring each package's name, and thus its enclosing directory, uniquely describes its purpose. While the symmetry with `cmd/` for package main commands is appealing, a directory that exists only to hold other packages is a potential design smell.

More importantly, the boilerplate of an empty `pkg/` directory distracts from the more useful idiom of an `internal/` directory. `internal/` is a special directory name recognised by the go tool which will prevent one package from being imported by another unless both share a common ancestor. Packages within an `internal/` directory are therefore said to be internal packages.

To create an internal package, place it within a directory named `internal/`. When the `go` command sees an import of a package

with `internal/` in the import path, it verifies that the importing package is within the tree rooted at the parent of the `internal/` directory.

For example, a package `/a/b/c/internal/d/e/f` can only be imported by code in the directory tree rooted at `/a/b/c`. It cannot be imported by code in `/a/b/g` or in any other repository.

If your project contains multiple packages you may find you have some exported symbols which are intended to be used by other packages in your project, but are not intended to be part of your project's public API. Although Go has limited visibility modifiers—public, exported, symbols and private, non exported, symbols—internal packages provide a useful mechanism for controlling visibility to parts of your project which would otherwise be considered part of its public versioned API.

You can, of course, promote internal packages later if you want to commit to supporting that API; just move them up a directory level or two. The key is this process is opt-in. As the author, internal packages give you control over which symbols in your project's public API without being forced to glob concepts together into unwieldy mega packages to avoid exporting them.

A bit of history. Read this post from 20 years ago by Phil Jones. That's what I was trying to do back then, just as Twitter came online. I didn't know it then but was the moment when the web stopped growing. When the VCs took over, and monetized the hell out of it. What we got in the end was Trump and Musk. We would have been smart, as a civilization, to hedge against the monopolies. If we get another chance what are we going to do with it? Will we work together this time? It's worth one more shot. My comments on the Jones piece in 2006 and 2026 .

Happy to report there are FeedLand users who want to edit OPML lists there so they can subscribe to them in another feed reader that has support for dynamic OPML lists. I am happy because this is a very cool feature that will be so much more fun if other people use it. If you want to set it up so you have a list on feedland.com that you want to subscribe to in another reader, instead of subscribing to all your feeds, like this – create a category for each list you want to hook up to another reader. It will be much easier to manage down the road. Categories in FeedLand are very simple, but if you use them carefully, they really help. Here's a screen shot of my Cats menu to give you an idea. I really use FeedLand in the most powerful ways, but it'll really click when others do the same.

We might be there now.

Podcast: “Accessibility on Apple’s platforms”, with special guest Sommer Panage

Swift by Sundell

Sommer Panage returns to the show to discuss Apple's various accessibility APIs and tools, how to incorporate accessibility support into a team's overall development workflow, and what it was like being an engineering manager at Apple.

Emerge Tools : Optimize your app's startup time, binary size, and overall performance using Emerge's advanced app optimization and monitoring tools. Get started at emergetools.com .

Bitrise : Rock-solid continuous integration for your Swift project, which now offers 50% faster builds and ad-ons for things like automatic deployment. Go to bitrise.io/swift to get started for free.

VoiceOver

PSPDFKit's blog post about Dynamic Type

Defining accessibility labels

Accessibility traits

WWDC session about custom accessibility actions

WWDC session about using AXCustomContent

isReduceMotionEnabled

prefersCrossFadeTransitions

shouldDifferentiateWithoutColor

SwiftUI's accessibilityElement modifier

Audio graphs

ANALYSIS

Jamie Cuevas, OS X Daily: If you're an iPhone user in the UK, you might want to hold off on the latest iOS 26.4 update for a bit, as many UK users are reporting problems with the new mandatory online Age Verification system that is included in the latest versions of iOS. Failures and issues with the age verification system ... [Read More](#)

Kelly Kirwan, Twilio Blog: Collecting data is easy. Activating it is hard. This guide covers the data activation process, common obstacles, and how to get it right.

Scripting News, Scripting News: I have a hard and fast rule about phone calls that solicit private information. I hang up. The worst are insurance companies. They expect you to enter all kinds of confidential info on a phone from a number that doesn't even verify as belonging to the company. Caller ID has nothing to say about them. Yet at least some of these are legit and unless you do what they want, you don't get your meds.

Scripting News, Scripting News: Send this video to your favorite Democrat and let them know that we would pay money to have this video run as an ad running everywhere, exactly as-is, no editing, not made glamorous. This is the truth that absolutely is not getting out about the law the Repubs want. We need to communicate with each other using the amazing tools we have at our disposal now in the third decade of the freaking 21st century.

Either way, the team must strive to work towards the greater vision, and consider that the product itself is just a means for achieving a greater goal.

Yann Lechelle

The Frontier Gazette

Vol. 1, No. 048 · 2026-03-30

Curated and composed automatically.